

Introducere în programare - curs și laborator - suport electronic

(C) 2023-2024 Bogdan Pătruț

Meniu

[Lectia 1](#) | [Lectia 2](#) | [Lectia 3](#) | [Lectia 4](#) | [Lectia 5](#) | [Lectia 6](#) | [Lectia 7](#)
[Material pentru laboratoare](#)

Lectia 5 - Fișiere

Scopul acestui curs este familiarizarea studenților cu lucrul cu fișiere text și binare în limbajul C. Laboratorul prezintă mai multe exemple practice de utilizare a fișierelor. De asemenea, se va vorbi despre utilizarea parametrilor din linia de comandă. **Video**

1. Prima parte (21:06) - fișiere1: [click aici](#)
2. A doua parte (13:35) - fișiere2: [click aici](#)
3. A treia parte (16:03)- fișiere3: [click aici](#)
4. A patra parte (9:04) - fișiere4: [click aici](#)
5. A cincea parte (20:15) - fișiere5: [click aici](#)
6. Utilizarea parametrilor din linia de comandă(13:42) : [click aici](#)

5.1. Introducere

În acest curs, veți afla despre prelucrarea fișierelor în limbajul C. Veți învăța să gestionați intrările și ieșirile standard în C folosind funcțiile `fprintf ()`, `fscanf ()`, `fread ()`, `fwrite ()`, `fseek ()` etc., cu ajutorul unor exemple. Un fișier este un container din dispozitivele de stocare a calculatorului, utilizate pentru stocarea datelor.

De ce este nevoie de fișiere?

Când un program este încheiat, toate datele sunt pierdute. Stocarea într-un fișier va păstra datele dumneavoastră chiar dacă programul se încheie. Dacă trebuie să introduceți un număr mare de date, va fi nevoie de mult timp pentru a le introduce pe toate. Cu toate acestea, dacă aveți un fișier care conține toate datele, puteți accesa cu ușurință conținutul fișierului folosind câteva comenzi din C. Puteți muta cu ușurință datele voastre de la un calculator la altul fără modificări.

5.2. Reprezentarea textuală vs binară a datelor

Fișierele sunt fișiere și, după cum am explicat și în videoclipurile asociate, au o reprezentare pe octeți/caractere, indiferent de conținutul lor. De multe ori lumea folosește termenii de „fișiere text”, respectiv „fișiere binare”, dar NU există două tipuri (text, respectiv binare) de fișiere, ci felul în care interpretăm conținutul acelui fișier.

Așadar, acest lucru depinde de modul în care datele sunt reprezentate în acel fișier: textual sau binar. Așa după cum am arătat și în videoclipuri, reprezentarea „binară” este cea în care date sunt văzute ca secvențe de octeți de lungime constantă, necesară stocării în memoria internă a acelei date. Reprezentarea textuală este cea în care datele sunt vizibile și interpretate de ochiul uman, în programe cum ar fi Notepad.

▼ Exemplificări cu explicații

- De pildă, caracterul A (care e o dată de tip `char`) este stocat în memorie printr-un octet, care reprezintă codul ASCII al acelui caracter, în cazul lui A fiind 65 (în baza 10), respectiv 41 (în hexazecimal/baza 16).
- Un șir de caractere din C/C++ (de exemplu "ABC") se codifică în memorie prin secvența octeților reprezentând codurile ASCII ale caracterelor ce compun acel șir, urmată de un octet cu valoarea 0, care reprezintă sfârșitul acelui șir (de fapt aceasta este o convenție pe care o folosim noi în C++, când lucrăm cu funcțiile din `string.h`). Lungimea acestei secvențe va fi de 4 octeți, din care 3 pentru fiecare caracter din șir, iar unul pentru caracterul cu codul ASCII 0, care marchează finalul șirului.
- Un număr întreg (de tip `int`) se stochează în memorie folosind codificarea prin complement față de 2, lungimea reprezentării fiind aceeași, (`sizeof(int)`, de exemplu 4). Așadar, numerele 12 și 123456789 se vor codifica tot pe 4 octeți, chiar dacă 12 este mai „scurt” decât 12345678. La o reprezentare textuală, evident că numărul 12 va fi reprezentat pe 2+1 octeți, pe când numărul 123456789 pe 9+1 octeți, deoarece vor fi văzute ca șiruri de caractere. Dar în reprezentarea „binară”, numerele întregi vor fi stocate pe `sizeof(int)` octeți, indiferent de valoarea lor sau de numărul lor de cifre („lungimea” lor). Trebuie precizat faptul că reprezentarea binară concretă din memorie a acelor octeți depinde de arhitectura hardware. (Există două modalități numite „little-endian” și „big-endian”; de exemplu calculatoarele x86/x64 folosesc ordinea „little-endian”, dar aceste lucruri sunt prea tehnice și nu fac subiectul cursului de față.
- Tipul `float` și tipul `double` desemnează numere raționale. Chiar dacă spunem că sunt numere reale, numerele iraționale nu se mai termină, deci noi putem reprezenta în calculator doar numere raționale. Ele se stochează folosind așa-numita reprezentare în virgulă mobilă, cu o anumită precizie. Două numere reale, de exemplu 1,456 (notat 1.456) și 3456,20395 (notat 3456.20395) se reprezintă tot pe același număr de octeți (ce poate fi obținut apelând la funcția `sizeof`). În reprezentarea „textuală”, evident că numărul de octeți va fi determinat de câte caractere sunt necesare pentru a-l scrie.

Prin reprezentare „textuală” înțelegem modul de introducere/afișare de la/către un periferic I/O a valorilor specifice unui anumit tip de date. Astfel, pentru introducerea unei valori de la tastatură, vorbim de secvența de taste apășate pentru introducerea acelei valori. Ceea ce se transmite între tastatură și procesor este o secvență de caractere. Similar, pentru afișarea unei valori pe un ecran în mod text (nu grafic), ceea ce se transmite dinspre unitatea centrală către placa grafică/ecran, este o secvență de caractere (reprezentate prin codurile lor ASCII), care apoi sunt "transformate" în imaginea "tipografică" a acelor caractere (litere, cifre, și altele), pe ecran.

▼ Exemplificări cu explicații

- numerele întregi se reprezintă „textual” printr-o secvență de cifre în baza 10, precedată eventual de semnul + sau -, deci lungimea reprezentării poate diferi de la un număr la altul;
- numerele raționale se reprezintă „textual” tot printr-o secvență de cifre în baza 10, dar de data aceasta, pe lângă + sau -, putem avea și separatorul zecimal (',' sau '.'); de asemenea, după cum știți din cursurile 2-3, o altă reprezentare este cea „științifică”, care folosește litera 'e' pentru a preciza exponentul lui 10 din reprezentarea sa (de pildă $2.57e+4$ înseamnă 2.57×10^4 , adică 25700).
- tipurile caracter și șir de caractere se reprezintă textual prin ele însele.

Atenție să nu confundați reprezentarea „binară” a datelor cu reprezentarea lor „textuală” în baza 2 (folosind cifrele 0 și 1). Așa după cum am arătat și în videoclipuri, la nivelul sistemului de fișiere, conținutul unui fișier este doar o secvență de octeți și depinde doar de noi să decidem cum interpretăm conținutul unui fișier, folosind reprezentarea „binară” sau cea „textuală”.

De regulă, numim fișier text un fișier pentru care s-a decis utilizarea reprezentării „textuale” la introducerea informației stocate în el. Acest lucru s-a realizat prin utilizarea unui editor de texte obișnuit (de exemplu Notepad) sau prin folosirea unui program oarecare, care salvează informațiile în fișier folosind reprezentarea „textuală”. Acest lucru se poate realiza în C apelând la funcția `fprintf()`.

Fișierele text au avantaje, cum ar fi că le putem citi ușor și edita după cum dorim, de exemplu în Notepad. Totuși, dezavantajele țin de securitate și de faptul că necesită un spațiu de stocare mai mare pentru date. De obicei, folosim anumite extensii pentru numele fișierelor text, prin care le împărțim astfel în subcategorii distincte de texte (de exemplu, extensia .cpp pentru fișiere cu cod sursă CPP, extensia .txt pentru fișiere create în Notepad, extensia .html pentru fișiere cu cod HTML etc., dar toate sunt fișiere text).

Când folosim reprezentarea „binară” pentru stocarea informației, acele fișiere le numim „binare”. Stocarea

datelor într-un fișier binar se face printr-un program creat de un programator, care folosește funcția de scriere `fwrite` sau de o firmă oarecare, care salvează informațiile în fișier folosind o anumită reprezentare „binară”, într-un anumit format propriu cunoscut doar de firma aceea sau un format public, pentru care se poate găsi o documentație.

De obicei, folosim anumite extensii pentru numele fișierelor „binare”, prin care le împărțim astfel în subcategorii distincte de informații „binare” (de exemplu: extensiile `.doc` și `.docx`, pentru fișiere de text formatat din Microsoft Word, `.xls` și `.xlsx` pentru fișiere de calcul tabelar în formatul Microsoft Excel, extensia `.exe` pentru fișiere cu cod executabil pe platforma Windows ș.a.). Asta nu înseamnă că dacă un fișier are o anumită extensie conținutul său chiar este de acel tip.

Fișierele binar nu se pot citi cu ușurință și oferă o mai bună securitate decât fișierele text, prin modalitatea de codificare folosită.

5.3. Lucrul cu fișiere

Când lucrați cu fișiere, trebuie să declarați un pointer/indicator/descriptor de tip fișier. Această declarație este necesară pentru comunicarea dintre fișier și program:

```
FILE *fptr;
```

În C, puteți efectua patru operații majore pe fișiere, fie text sau binar:

1. crearea unui nou fișier;
2. deschiderea unui fișier existent;
3. închiderea unui fișier;
4. citirea și scrierea informațiilor într-un fișier.

5.4. Deschiderea unui fișier - pentru creare și editare

Deschiderea unui fișier este realizată folosind funcția `fopen()`, definită în fișierul header `stdio.h`. Sintaxa pentru deschiderea unui fișier standard, în I/O este:

```
ptr = fopen("filename", "mode");
```

De exemplu,

```
fopen("E:\\cprogram\\newprogram.txt", "w");  
fopen("E:\\cprogram\\oldprogram.bin", "rb");
```

Să presupunem că fișierul `newprogram.txt` nu există în locația `E:\\cprogram`. Prima funcție creează un nou fișier numit `newprogram.txt` și îl deschide pentru scriere în modul `'w'`. Modul de scriere vă permite să creați și să editați (suprascrieți) conținutul fișierului. Acum să presupunem că al doilea fișier binar `oldprogram.bin` există în locația `E:\\cprogram`. A doua funcție deschide fișierul existent pentru citire în modul binar `'rb'`. Modul de citire vă permite doar să citiți fișierul, nu puteți scrie în fișier.

5.4.1. Moduri de deschidere a fișierelor în I/O standard

Moduri de deschidere în Standard I/O

Mod	Semnificație	În timpul inexistenței fișierului of file
<code>r</code>	Deschis pentru citire.	Dacă fișierul nu există, <code>fopen()</code> returnează <code>NULL</code> .

Mod	Semnificație	În timpul inexistenței fișierului of file
<code>rb</code>	Deschis pentru citire în mod binar.	Dacă fișierul nu există, <code>fopen()</code> returnează <code>NULL</code> .
<code>w</code>	Deschis pentru scriere.	Dacă fișierul există, conținutul său este suprascris. Dacă fișierul nu există, va fi creat.
<code>wb</code>	Deschis pentru scriere în mod binar.	Dacă fișierul există, conținutul său este suprascris. Dacă fișierul nu există, va fi creat.
<code>a</code>	Deschis pentru adăugare (<i>append</i>). Datele sunt adăugate la finalul fișierului.	Dacă fișierul nu există, atunci el va fi creat.
<code>ab</code>	Deschis pentru adăugare în mod binar. Datele sunt adăugate la finalul fișierului.	Dacă fișierul nu există, atunci el va fi creat.
<code>r+</code>	Deschis atât pentru citire, cât și pentru scriere.	Dacă fișierul nu există, <code>fopen()</code> returnează <code>NULL</code> .
<code>rb+</code>	Deschis atât pentru citire, cât și pentru scriere în mod binar.	Dacă fișierul nu există, <code>fopen()</code> returnează <code>NULL</code> .
<code>w+</code>	Deschis atât pentru citit, cât și pentru scris.	Dacă fișierul există, conținutul său este suprascris. Dacă fișierul nu există, atunci el va fi creat.
<code>wb+</code>	Deschis atât pentru citire, cât și scriere, în mod binar.	Dacă fișierul există, conținutul său va fi suprascris. Dacă fișierul nu există, atunci el va fi creat.
<code>a+</code>	Deschis atât pentru citire, cât și pentru adăugare.	Dacă fișierul nu există, atunci va fi creat.
<code>ab+</code>	Deschis atât pentru citire, cât și pentru adăugare, în mod binar..	Dacă fișierul nu există, atunci el va fi creat.

5.4.2. Închiderea unui fișier

După citire/scriere, un fișier (fie text, fie binar) trebuie închis. Închiderea unui fișier se realizează folosind funcția `fclose()`, astfel:

```
fclose(fp);
```

Aici, `fp` este un pointer/indicator/descriptor asociat fișierului ce se închide.

5.5. Citirea și scrierea dintr-un/într-un fișier text

Pentru citirea și scrierea dintr-un/într-un fișier text, folosim funcțiile `fprintf()` și `fscanf()`. Sunt doar versiunile de fișiere ale funcțiilor cunoscute `printf()` și `scanf()`. Singura diferență este că `fprint()` și `fscanf()` așteaptă un indicator la structura `FILE`.

▼ Exemplul 1: Scrierea într-un fișier text

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int num;
    FILE *fp;
    // use appropriate location if you are using MacOS or Linux
```

```

        fptr = fopen("C:\\program.txt","w");
        if(fptr == NULL)
        {
            printf("Error!");
            exit(1);
        }
        printf("Enter num: ");
        scanf("%d",&num);
        fprintf(fptr,"%d",num);
        fclose(fptr);
        return 0;
    }

```

Acest program preia un număr de la utilizator și îl stochează în fișierul `program.txt`. După ce compilați și rulați acest program, puteți vedea un fișier text `program.txt` creat în unitatea de disc C a computerului. Când deschideți fișierul, puteți vedea numărul întreg pe care l-ați introdus.

▼ Exemplul 2: Citirea dintr-un fișier text

```

#include <stdio.h>
#include <stdlib.h>
int main()
{
    int num;
    FILE *fptr;
    if ((fptr = fopen("C:\\program.txt","r")) == NULL){
        printf("Error! opening file");
        // Program exits if the file pointer returns NULL.
        exit(1);
    }
    fscanf(fptr,"%d", &num); printf("Value of n=%d", num); fclose(fptr);
    return 0;
}

```

Acest program citește întregul prezent în fișierul `"program.txt"` și îl afișează pe ecran. Dacă ați creat cu succes fișierul din Exemplul 1, executarea acestui program vă va obține numărul întreg pe care l-ați introdus. Alte funcții precum `fgetchar()`, `fputc()` etc. pot fi utilizate într-un mod similar.

5.6. Citirea și scrierea din/în fișiere binare

Funcțiile `fread()` și `fwrite()` sunt utilizate pentru citirea și scrierea dintr-un/într-un fișier de pe disc, în cazul fișierelor binare.

5.6.1. Scrierea într-un fișier binar

Pentru a scrie într-un fișier binar, trebuie să utilizați funcția `fwrite()`. Funcțiile au patru argumente:

1. adresa datelor care trebuie scrise pe disc;
2. dimensiunea datelor care trebuie scrise pe disc;
3. numărul de astfel de date;
4. pointer către fișierul în care doriți să scrieți.

```
fwrite(addressData, sizeData, numbersData, pointerToFile);
```

▼ Exemplul 3: Scrierea într-un fișier binar folosind `fwrite()`

```

#include <stdio.h>
#include <stdlib.h>

struct threeNum
{
    int n1, n2, n3;
};
int main()
{
    int n;
    struct threeNum num;
    FILE *fptr;
    if ((fptr = fopen("C:\\program.bin","wb")) == NULL){
        printf("Error! opening file");
        // Program exits if the file pointer returns NULL.
        exit(1);
    }
    for(n = 1; n < 5; ++n)
    {
        num.n1 = n;
        num.n2 = 5*n;
        num.n3 = 5*n + 1;
        fwrite(&num, sizeof(struct threeNum), 1, fptr);
    }
    fclose(fptr);
    return 0;
}

```

În acest program, noi am creat un fișier nou, cu numele "program.bin" în unitatea de disc C. Declarăm o structură `threeNum` cu trei numere, `n1`, `n2` și `n3`, și o definim în funcția `main` cu numele `num`. Acum, în interiorul buclei `for`, stocăm valoarea în fișier, folosind `fwrite()`.

Primul parametru ia adresa `num` și al doilea parametru ia dimensiunea structurii `threeNum`.

Deoarece introducem o singură instanță de `num`, al treilea parametru este 1. Și ultimul parametru `* fptr` indică fișierul pe care îl stocăm.

La final închidem fișierul.

5.6.2. Citirea dintr-un fișier binar

Funcția `fread()` folosește 4 argumente similare funcției `fwrite()` de mai sus.

```
fread(addressData, sizeData, numbersData, pointerToFile);
```

▼ Exemplul 4: Citirea dintr-un fișier binar folosind `fread()`

```

#include <stdio.h>
#include <stdlib.h>
struct threeNum
{
    int n1, n2, n3;
};
int main()
{
    int n;
    struct threeNum num;
    FILE *fptr;
    if ((fptr = fopen("C:\\program.bin","rb")) == NULL){
        printf("Error! opening file");
        // Program exits if the file pointer returns NULL.
        exit(1);
    }
    for(n = 1; n < 5; ++n)

```

```

    {
        fread(&num, sizeof(struct threeNum), 1, fptr);
        printf("n1: %d\\tn2: %d\\tn3: %d", num.n1, num.n2, num.n3);
    }
    fclose(fptr);
    return 0;
}

```

În acest program, citiți același fișier program.bin și parcurgeți înregistrările una câte una. În termeni simpli, citiți o înregistrare threeNum de dimensiunea lui threeNum din fișierul indicat de *fptr în structura num. Veți obține aceleași înregistrări pe care le-ați inserat în exemplul anterior.

5.7. Obținerea de date folosind **fseek()**

Dacă aveți multe înregistrări într-un fișier și trebuie să accesați o înregistrare într-o anumită poziție, trebuie să faceți o buclă prin toate înregistrările înainte de a obține înregistrarea. Acest lucru va pierde multă memorie și timp de funcționare. O modalitate mai ușoară de a ajunge la datele solicitate poate fi obținută folosind **fseek()**. După cum sugerează și numele, **fseek()** caută cursorul la înregistrarea dată în fișier.

Sintaxa lui **fseek()**

```
fseek(FILE * stream, long int offset, int whence);
```

Primul parametru este pointerul către fișier. Al doilea este poziția înregistrării ce trebuie să fie găsită, iar al treilea parametru specifică locul de unde offsetul începe.

Diferiți parametri „whence” în **fseek()**

1. **SEEK_SET** -Începe offset-ul de la începutul fișierului.
2. **SEEK_END** -Începe offset-ul de la finalul fișierului.
3. **SEEK_CUR** -Începe offset-ul de la poziția curentă a cursorului din fișier.

▼ Exemplul 5: **fseek()**

```

#include <stdio.h>
#include <stdlib.h>
struct threeNum
{
    int n1, n2, n3;
};
int main()
{
    int n;
    struct threeNum num;
    FILE *fptr;
    if ((fptr = fopen("C:\\program.bin","rb")) == NULL){
        printf("Error! opening file");
        // Program exits if the file pointer returns NULL.
        exit(1);
    }
    // Moves the cursor to the end of the file
    fseek(fptr, -sizeof(struct threeNum), SEEK_END);
    for(n = 1; n < 5; ++n)
    {
        fread(&num, sizeof(struct threeNum), 1, fptr);
        printf("n1: %d\\tn2: %d\\tn3: %d\\n", num.n1, num.n2, num.n3);
        fseek(fptr, -2*sizeof(struct threeNum), SEEK_CUR);
    }
    fclose(fptr);
}

```

```
return 0;  
}
```

Acest program va începe să citească înregistrările din fișierul program.bin în ordine inversă (de la ultimul până la primul) și îl va afișa.

5.8. Programe demonstrative

Sunt disponibile [aici](#) și au fost trimise pe serverul Discord asociat disciplinei.